

Week 2 Day 3 Academic And Professional Foundations

Day 3는 image, Dockerfile, build context, cache, registry를 다룬다. Day 2가 실행 중 state와 연결 관계였다면, Day 3는 실행 가능한 artifact를 어떻게 만들고 식별하고 공유할지에 초점을 둔다.

기준	Day 3 연결
Docker image docs	image, layer, tag, digest, immutable artifact 개념
Dockerfile reference	FROM, WORKDIR, COPY, RUN, CMD의 공식 동작
Docker build docs	build context, cache, layer, .dockerignore의 영향
Docker image tag/history	tag와 layer history를 확인
Docker Hub docs	registry push/pull 흐름과 credential 보호
OCI Image Specification	manifest, layer, digest를 content-addressed artifact로 이해
OWASP Secrets Management	secret을 build context/image/registry에 포함하지 않는 기준

Conceptual Rationale

Image는 container를 실행하기 위한 표준 패키지다. 학생은 "내 컴퓨터에서 되는 코드"를 Dockerfile과 build context로 고정하고, build 결과를 tag와 image ID로 식별한다. tag는 사람이 쓰기 쉬운 이름이고 digest는 content identity에 가깝다는 차이를 함께 설명한다.

Dockerfile 수업의 위험은 문법 나열로 끝나는 것이다. Day 3에서는 매 instruction이 cache와 layer에 어떤 영향을 주는지 확인하고, .dockerignore가 없으면 어떤 파일이 build context에 들어갈 수 있는지 보여준다. registry는 마지막에 다루되, public push는 credential과 secret 점검을 통과한 경우에만 선택 실습으로 둔다.

Official Links

- What is an image?: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/>
- Dockerfile reference: <https://docs.docker.com/reference/dockerfile/>
- Writing a Dockerfile: <https://docs.docker.com/guides/docker-concepts/building-images/writing-a-dockerfile/>
- Docker build command: <https://docs.docker.com/reference/cli/docker/buildx/build/>
- Build context: <https://docs.docker.com/build/building/context/>
- Docker image tag: <https://docs.docker.com/reference/cli/docker/image/tag/>
- Docker image history: <https://docs.docker.com/reference/cli/docker/image/history/>

- Docker Hub: <https://docs.docker.com/docker-hub/>
- OCI Image Specification: <https://specs.opencontainers.org/image-spec/>
- OWASP Secrets Management Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Standards Crosswalk

기준	학생 행동
Bloom apply/analyze	Dockerfile을 만들고 build output에서 cache/layer/path 문제를 분석
ABET-style problem solving	build 실패를 context, path, instruction, permission 문제로 분류
Professional responsibility	secret과 불필요한 파일이 image나 registry에 들어가지 않게 확인
DevOps handoff	build/run/check/cleanup/troubleshoot 흐름을 재현 가능하게 설명

완료 전 주의할 점

- Build context에 secret, log, 큰 dependency directory가 들어가면 image size와 보안 위험이 커진다.
- Dockerfile instruction은 build 시점과 run 시점의 의미가 다르다. RUN, CMD, EXPOSE, -p를 섞지 않는다.
- tag는 사람이 읽기 쉬운 이름이고 digest는 content 식별자에 가깝다. 재현성이 필요하면 digest를 함께 본다.
- Registry push는 공개 범위와 credential 문제가 생긴다. push는 선택으로 두고 gate를 먼저 통과한다.
- build 실패는 Docker 전체 문제가 아니라 missing file, wrong CMD, wrong port, bloated context처럼 좁혀 볼 수 있다.